

Weather Prediction Market Trading Simulator

Piya Tewari

Contents

1	Introduction	2
2	Prediction Market Framework	2
2.1	Binary Weather Contracts	2
2.2	Fair Value Estimation	2
2.3	Edge Calculation	3
3	Trading System	3
4	Forecast Updating	4
4.1	Probability Paths	4
4.2	Bayesian Updating	5
5	Simulation Framework	5
5.1	Trade Logging	6
5.2	Contract Resolution	6
6	Performance Evaluation	7
6.1	Visualization	7
6.2	Monte Carlo Backtesting	7
6.3	Risk Metrics	8
6.4	Monte Carlo Results	8
7	Live Forecast Integration	8
7.1	Forecast-Based Fair Value	9
7.2	Live Trading Signals	9
8	Multi-City Scanner	10
8.1	Cross-Market Comparison	10
8.2	Scanner Output	11
9	Limitations	11
10	What I Learned	12
11	Conclusion	12

1 Introduction

In prediction markets, contracts trade based on the outcome of future events. In a binary one, a YES contract pays \$1 if the event occurs and \$0 otherwise. Because of this payoff structure, the price of a YES contract can be interpreted as the market's implied probability of the event occurring.

This project builds a weather-focused prediction market trading simulator in Python. The objective is to model how a trader might value, trade, and manage risk in binary weather event contracts. The project combines probabilistic forecasting, fair value estimation, trading signals, position sizing, inventory management, Bayesian updating, Monte Carlo simulation, and live weather forecast data.

The main contract studied throughout development was: **Will Philadelphia receive more than 0.50 inches of rain tomorrow?**

The project expanded from a single Philadelphia rainfall contract into a multi-city weather prediction market scanner covering Philadelphia, New York, Chicago, and Miami.

2 Prediction Market Framework

2.1 Binary Weather Contracts

The first part of the simulator was a binary weather contract object. Each contract represents a weather event, such as rainfall exceeding a threshold in a specific city. The contract stores the event description, city, threshold, contract type, resolution status, and final outcome.

A YES contract pays

$$\begin{cases} 1, & \text{if the event occurs,} \\ 0, & \text{otherwise.} \end{cases}$$

A NO contract pays

$$\begin{cases} 0, & \text{if the event occurs,} \\ 1, & \text{otherwise.} \end{cases}$$

This structure created the foundation for later components including fair value estimation, trading signals, inventory management, and event resolution.

2.2 Fair Value Estimation

Prediction market prices can be interpreted as probabilities. If the model estimates that an event has probability p of occurring, then the fair value of a YES contract is

$$p.$$

Similarly, the fair value of a NO contract is

$$1 - p.$$

For example, if the model estimates a 72% chance that Philadelphia receives more than 0.50 inches of rain, then the fair value of the YES contract is

0.72.

Throughout the project, forecast probabilities were converted into fair values and compared against market prices to identify potential trading opportunities.

2.3 Edge Calculation

After calculating fair value, the model compares its forecast-implied probability to the market price. The model edge is defined as

$$\text{Model Probability} - \text{Market Price}.$$

If the result is positive, the model believes the contract is underpriced. If the result is negative, the model believes the contract is overpriced.

This edge calculation moved the project from pure forecasting into prediction-market trading logic.

3 Trading System

After building the contract structure and fair value calculations, the next step was creating a trading system. The goal was to allow the model to compare its forecast against the market price and decide whether a trade should occur.

The first version of the trading system generated simple trading signals. If the model believed the contract was underpriced, it generated a BUY YES signal. If the model believed the contract was overpriced, it generated a SELL YES signal. If the difference between the model estimate and the market price was small, the model did not trade.

The model uses edge to determine whether a trading opportunity exists. A positive edge indicates that the model believes the contract is too cheap, while a negative edge indicates that the model believes the contract is too expensive.

Once trading signals were introduced, the simulator needed a way to track positions and profits. A portfolio object was added to keep track of cash, inventory, portfolio value, and profit and loss.

When the trader buys YES contracts, cash decreases and inventory increases. When the trader sells YES contracts, cash increases and inventory decreases. The portfolio value is continuously updated as market prices change.

Profit and loss is calculated as

$$\text{Portfolio Value} - \text{Initial Capital}.$$

At this stage the simulator could generate trading signals, execute trades, track inventory, and monitor profitability through time.

The next improvement was position sizing. The earliest version of the simulator traded a single contract whenever a signal was generated. While useful for testing, this approach treated every opportunity equally.

To make the system more realistic, trade size was linked to the size of the edge. Larger edges represent stronger conviction and therefore lead to larger positions.

This allowed the model to allocate more capital when it believed the market was substantially mispriced and less capital when the advantage was small.

The final component of the trading system was risk management. Inventory limits were introduced to prevent unlimited directional exposure.

For example, a trader operating in conservative mode is not allowed to establish short positions. Even if the model generates a SELL YES signal, the trade can still be blocked if it would violate inventory limits.

This created an important separation between signal generation and execution. The forecasting model identifies opportunities, but the risk management system determines whether those opportunities can actually be traded.

By the end of this stage, the project had evolved from a simple forecasting tool into a complete trading simulator capable of generating signals, managing positions, enforcing risk limits, and tracking performance.

4 Forecast Updating

After building the trading system, the next challenge was making the forecast itself more realistic. In the earliest version of the simulator, the model probability remained fixed throughout the entire simulation. While this was useful for testing the trading engine, it did not reflect how real weather forecasts evolve as new information becomes available.

To address this, the simulator was extended to include weather information updates. These updates represent new pieces of information arriving over time, such as radar observations, forecast revisions, or changing weather conditions.

Three types of updates were introduced:

- Bullish updates
- Bearish updates
- Neutral updates

A bullish update increases confidence that the event will occur. A bearish update decreases confidence that the event will occur. A neutral update has little effect on the forecast.

At each time step, the simulator generates a weather update and adjusts the model probability accordingly. This allows the forecast to evolve throughout the simulation instead of remaining fixed.

The introduction of weather updates created a more realistic environment because the trading model now had to react to changing information rather than relying on a single static forecast.

4.1 Probability Paths

Once weather updates were introduced, the simulator began generating a sequence of forecast probabilities rather than a single probability estimate.

For example, a simulation might begin with a forecast probability of

0.72.

After several bearish updates, the probability might fall below its initial value. Alternatively, a sequence of bullish updates could increase confidence that the event will occur.

As a result, the model probability becomes a path that evolves through time.

This probability path directly influences trading decisions because fair value is recalculated after every update. As the forecast changes, the model may switch from buying to selling or from selling to buying.

The probability path therefore became one of the most important drivers of strategy behavior.

4.2 Bayesian Updating

The first version of weather updates used simple probability adjustments. For example, a bullish update might add a fixed amount to the forecast probability while a bearish update might subtract a fixed amount.

Although this approach was useful for testing, it was not statistically rigorous. The simulator was therefore extended to use Bayesian updating.

Bayesian updating provides a framework for revising probabilities when new information becomes available. Instead of manually increasing or decreasing the forecast, the model updates its belief based on the likelihood of observing a particular signal.

The posterior probability is calculated using Bayes' theorem.

$$\frac{P(\text{Signal} \mid \text{Event})P(\text{Event})}{P(\text{Signal} \mid \text{Event})P(\text{Event}) + P(\text{Signal} \mid \text{No Event})P(\text{No Event})}.$$

In this project, the prior probability represents the current forecast estimate. The weather update acts as the signal, and the resulting posterior probability becomes the new forecast.

This approach allowed the model to respond to new information in a more principled way. Strong bullish signals produced larger probability increases, while strong bearish signals produced larger probability decreases.

The transition from fixed probability adjustments to Bayesian updating was one of the most important improvements made during the project. It transformed the forecasting component from a simple rule-based system into a probabilistic model that incorporates new evidence over time.

By the end of this stage, the simulator contained a dynamic forecasting process that continuously updated probabilities, recalculated fair values, and generated new trading opportunities as information arrived.

5 Simulation Framework

Once forecasting and trading components were working independently, the next step was combining them into a complete simulation framework.

The purpose of the simulation framework is to model how a prediction market trader would behave over time as market prices change and new information arrives. Instead of evaluating a single market price, the simulator generates an entire sequence of prices and forecast updates.

At each time step, the simulator performs the following actions:

1. Receive a market price.
2. Receive a weather update.
3. Update the forecast probability.
4. Calculate fair value.
5. Calculate edge.

6. Generate a trading signal.
7. Determine position size.
8. Check risk limits.
9. Execute a trade if allowed.
10. Update portfolio value and profit and loss.

This process is repeated until the contract reaches expiration.

The simulation framework transformed the project from a collection of individual components into a complete trading system capable of making decisions through time.

5.1 Trade Logging

As the simulator became more complex, it became increasingly important to record what happened during each run.

A trade log was introduced to store information at every time step. Each entry records the market price, forecast probability, weather update, edge, signal, position size, inventory level, cash balance, portfolio value, and profit and loss.

This information allows the entire simulation to be reconstructed after it finishes.

The trade log also makes it possible to diagnose unexpected behavior. If a trade occurs when it should not have occurred, or if a signal appears incorrect, the log provides a detailed record of the state of the system at that point in time.

As the project expanded, the trade log became the primary source of information for performance analysis and debugging.

5.2 Contract Resolution

Prediction market contracts eventually resolve to either YES or NO. A realistic simulator therefore needs a mechanism for determining the final outcome of the contract.

At expiration, the contract is resolved using the realized rainfall amount. If the rainfall exceeds the specified threshold, the YES contract pays 1. Otherwise, the YES contract pays 0.

The final value of the trader's position is therefore determined by both the inventory held at expiration and the contract payout.

The value of the position is

$$\text{YES Inventory} \times \text{YES Final Payout}.$$

The final portfolio value is then calculated by combining the remaining cash balance and the final value of the contracts held.

Profit and loss is determined by comparing the final portfolio value to the initial capital.

This stage completed the trading lifecycle. The simulator could now open positions, manage those positions through time, and ultimately settle them when the event outcome became known.

6 Performance Evaluation

Once the simulator could complete a full trading cycle, the next objective was measuring performance.

A single simulation can provide useful intuition, but it does not reveal whether a strategy is consistently effective. Performance evaluation was therefore introduced to summarize results across an entire simulation.

Several statistics were added to track the behavior of the strategy.

- Total trades
- Blocked trades
- Maximum inventory
- Average edge
- Final profit and loss

These metrics provide insight into how aggressively the strategy trades, how often risk limits become active, and whether the model is successfully identifying profitable opportunities.

6.1 Visualization

Visualizations were added to make the behavior of the simulator easier to understand.

The first plot compares market prices against model probabilities throughout the simulation. This makes it possible to see when the model disagrees with the market and when trading opportunities appear.

A second plot tracks inventory through time. This shows how the trader accumulates and reduces positions as conditions change.

A third plot tracks profit and loss. This provides a visual summary of strategy performance and highlights periods where gains or losses occur.

These visualizations proved extremely useful for debugging and understanding how different components of the simulator interact.

6.2 Monte Carlo Backtesting

A single simulation path provides only one possible outcome. To evaluate the robustness of the strategy, the simulator was extended into a Monte Carlo backtesting framework.

Instead of running the strategy once, the simulator generates many different market environments using different random seeds.

Each simulation produces a different sequence of market prices, weather updates, forecast probabilities, and trading decisions.

By running the strategy repeatedly, it becomes possible to study the distribution of outcomes rather than a single result.

This is important because trading performance is inherently uncertain. A strategy that performs well in one simulation may perform poorly in another.

Monte Carlo simulation provides a more complete picture of risk and expected performance.

6.3 Risk Metrics

After running many simulations, several summary statistics are calculated.

The average final profit and loss measures the expected profitability of the strategy.

The standard deviation of profit and loss measures variability in outcomes.

The win rate measures the fraction of simulations that end profitably.

A Sharpe-like ratio is calculated by dividing average profit and loss by the standard deviation of profit and loss.

$$\frac{\text{Average Final PnL}}{\text{PnL Standard Deviation}}.$$

Although this is not a traditional Sharpe ratio, it provides a simple measure of return relative to risk.

Together, these metrics provide a more complete picture of strategy quality than final profit and loss alone.

6.4 Monte Carlo Results

After implementing Bayesian updating and probabilistic contract resolution, the simulator produced more realistic performance distributions.

The strategy remained profitable on average, but losses were now possible. This was an important improvement because it prevented the simulator from producing unrealistically optimistic results.

The final Monte Carlo analysis reported the average profit and loss, win rate, standard deviation of profit and loss, Sharpe-like ratio, and the best and worst simulation outcomes.

A histogram of final profit and loss was also generated to visualize the distribution of results.

This stage completed the analytical portion of the project and provided a framework for evaluating future improvements to the trading strategy.

7 Live Forecast Integration

Up to this point, all weather information in the simulator had been generated synthetically. While this was useful for developing and testing the trading system, it did not reflect how a real prediction market trader would obtain information.

To make the project more realistic, live weather forecast data was integrated into the forecasting process. Instead of relying entirely on simulated updates, the model now uses actual weather forecasts to estimate the probability of a rainfall event.

The forecast data is retrieved using a weather API. For a given city, the API provides hourly weather forecasts including precipitation probabilities.

The simulator extracts the hourly precipitation probabilities and stores them as a forecast path. These values represent the model's estimate of rainfall likelihood throughout the forecast horizon.

For example, a forecast might produce a sequence of hourly probabilities such as

$$0.01, 0.02, 0.03, 0.02, 0.01.$$

The simulator then analyzes these probabilities to estimate the likelihood of the contract event occurring.

Several summary statistics are calculated from the forecast data.

- Maximum rain probability
- Average rain probability
- Time of maximum probability

These statistics provide a simplified representation of the forecast while preserving the most important information.

Integrating live weather data represented a major milestone because it connected the simulator to real-world information. The forecasting component was no longer purely synthetic and could now react to actual weather conditions.

7.1 Forecast-Based Fair Value

Once live weather forecasts became available, the simulator used them to estimate contract fair value.

For example, suppose the maximum forecasted probability of rain is

$$0.03.$$

The simulator interprets this value as the model's estimate of the event probability.

The fair value of the YES contract therefore becomes

$$0.03.$$

If the market price exceeds this value, the contract appears overpriced. If the market price falls below this value, the contract appears underpriced.

This process allows the simulator to convert weather information into trading decisions.

7.2 Live Trading Signals

After calculating fair value, the simulator compares the forecast-implied probability against the market price.

For example, a forecast might produce

$$0.03$$

while the market price is

$$0.10.$$

The resulting edge is

$$0.03 - 0.10 = -0.07.$$

Because the edge is negative, the model believes the contract is overpriced and generates a SELL YES signal.

This completed the transition from simulated forecasting to real forecast-driven trading decisions.

8 Multi-City Scanner

Once live forecast integration was working for a single city, the next step was expanding the system to evaluate multiple markets simultaneously.

The simulator was extended to analyze weather contracts across several cities.

The initial scanner evaluated

- Philadelphia
- New York
- Chicago
- Miami

For each city, the scanner retrieves weather forecast data, estimates a fair value, compares that value against the market price, and generates a trading signal.

This process mirrors how a professional trader might search across many markets looking for opportunities rather than focusing on a single contract.

The scanner produces a summary for each city containing forecast statistics, fair value estimates, trading signals, and position recommendations.

For example, one city may produce

BUY YES

while another produces

SELL YES.

The scanner therefore allows the strategy to identify relative opportunities across multiple geographic regions.

8.1 Cross-Market Comparison

One advantage of scanning multiple cities is that it becomes possible to compare opportunities directly.

During testing, different cities often produced dramatically different forecast probabilities.

For example, one city might have a maximum rain probability of

0.03

while another might have a probability of

0.87.

These differences naturally produce different fair values and therefore different trading signals.

By evaluating all cities simultaneously, the simulator can focus capital on markets where the estimated edge is largest.

This represents a significant improvement over a single-market approach.

8.2 Scanner Output

For each city, the scanner reports

- Maximum rain probability
- Average rain probability
- Time of maximum probability
- Market price
- Model fair value
- Edge
- Trading signal
- Suggested quantity
- Risk-management decision

These outputs provide a concise summary of the most relevant information for decision making.

At this stage, the project had evolved from a single-contract simulator into a system capable of identifying opportunities across multiple prediction markets.

9 Limitations

Although the simulator successfully models many aspects of prediction market trading, several limitations remain.

The first limitation is forecast quality. The strategy relies heavily on weather forecasts, and inaccurate forecasts can lead to poor trading decisions.

A second limitation is market realism. The simulator assumes that trades can be executed immediately at the observed market price. Real prediction markets often contain bid-ask spreads, transaction costs, and limited liquidity.

A third limitation is contract modeling. The project focuses primarily on binary rainfall contracts. Real prediction markets contain a much wider variety of event structures and payout mechanisms.

Another limitation is position sizing. The current sizing methodology is intentionally simple and primarily based on edge magnitude. Professional trading systems often use more sophisticated capital allocation methods.

The Monte Carlo framework also relies on assumptions about how market prices and forecast updates evolve. Although useful for testing, simulated market behavior can never perfectly replicate real-world conditions.

Finally, the project evaluates each market independently. In reality, weather events across cities may be correlated, creating opportunities and risks that are not currently captured by the simulator.

Despite these limitations, the project provides a useful framework for studying prediction market trading and demonstrates how forecasting, probability estimation, risk management, and market pricing can be combined into a unified trading system.

10 What I Learned

This project taught me much more than how prediction markets work. The most important lesson was learning how to combine forecasting, probability, trading logic, risk management, simulation, and data analysis into a single system.

At the beginning of the project, I viewed prediction markets primarily as betting markets. As I continued developing the simulator, I began to understand that prediction market trading is fundamentally an exercise in probability estimation. Every trading decision ultimately comes from comparing an estimated probability against a market-implied probability.

Building the trading system also demonstrated the importance of risk management. Early versions of the simulator focused entirely on identifying profitable opportunities. As the project evolved, it became clear that finding an edge is only part of the problem. Position sizing, inventory limits, and execution constraints play an equally important role in determining long-term performance.

Another important lesson involved uncertainty. A strategy that appears profitable in a single simulation may perform poorly under different conditions. The Monte Carlo backtesting framework highlighted the importance of evaluating an entire distribution of outcomes rather than focusing on a single result.

The introduction of Bayesian updating was particularly valuable because it provided a principled framework for incorporating new information. Instead of manually adjusting probabilities, the model could update beliefs in a mathematically consistent manner as new evidence arrived.

Working with live weather forecasts also demonstrated the challenges of real-world data. Forecasts are noisy, incomplete, and constantly changing. Integrating live information required additional data processing and forced the model to operate in a less predictable environment than the synthetic simulations used earlier in development.

Finally, this project reinforced the importance of iterative development. The final system was not designed all at once. Instead, it was built by gradually adding new components, testing them, identifying limitations, and improving the model over time. This process mirrored how quantitative trading systems are often developed in practice.

11 Conclusion

This project developed a weather-based prediction market trading simulator capable of forecasting event probabilities, generating trading signals, managing risk, simulating market activity, and evaluating performance through backtesting.

The project began with a simple binary weather contract and gradually expanded into a complete trading framework. Major components included fair value estimation, edge calculation, trade execution, position sizing, inventory management, Bayesian forecasting, Monte Carlo simulation, live weather forecast integration, and multi-city opportunity scanning.

The completed system demonstrates how probabilistic forecasts can be transformed into trading decisions. By comparing model-implied probabilities against market prices, the simulator identifies potential opportunities and evaluates their performance under a variety of market conditions.

Overall, the project successfully achieved its objective of creating an end-to-end prediction market trading simulator while providing valuable experience in forecasting, quantitative trading, risk management, simulation, and data-driven decision making.